

Hướng Dẫn Thực Hành - Lập Trình Web Với Node.js

Tài liệu hướng dẫn thực hành dành cho sinh viên. Mỗi bài gồm: mục tiêu, hướng dẫn từng bước, cách chạy, kết quả mong đợi và lỗi thường gặp.

Yêu cầu trước khi bắt đầu: Đã cài đặt Node.js (phiên bản 18 trở lên). Kiểm tra bằng lệnh:

```
node -v
npm -v
```

Nếu chưa cài, tải tại: <https://nodejs.org>

Mục Lục

- **[Chương 1: Giới Thiệu Node.js](#)**
 - [Bài 1.4: Tạo HTTP Server Cơ Bản](#)
- **[Chương 2: Các Module Built-in Của Node.js](#)**
 - [Bài 2.2-os: Module OS - Thông Tin Hệ Điều Hành](#)
 - [Bài 2.2-path: Module Path - Xử Lý Đường Dẫn](#)
 - [Bài 2.2-fs: Module File System - Đọc/Ghi File](#)
 - [Bài 2.th-01: Thực Hành Tổng Hợp - Đọc JSON & Ghi Báo Cáo](#)
- **[Chương 3: Express.js Cơ Bản](#)**
 - [Bài 3.1: Node Modules & NPM](#)
 - [Bài 3.3: Express, Morgan & Các Loại Tham Số Request](#)
 - [Bài 3.th: Thực Hành - Xây Dựng RESTful API CRUD Sinh Viên](#)
- **[Chương 4: Kiến Trúc MVC](#)**
 - [Bài 4.0: So Sánh HTTP Thuần vs Express](#)
 - [Bài 4.4-mvc: Giới Thiệu Kiến Trúc MVC](#)
 - [Bài 4.4.thuchanh: Thực Hành - API Users Đơn Giản \(Chưa MVC\)](#)
 - [Bài 4.4.thuchanh-2: Thực Hành - CRUD Đầy Đủ Với MVC \(Nhiều Resource\)](#)
- **[Chương 5: Template Engine - EJS](#)**
 - [Bài 5.2: Ứng Dụng Web Với EJS](#)
- **[Chương 6: Middleware](#)**
 - [Bài 6.2-code-log: Middleware Ghi Log Request](#)
 - [Bài 6.2-code-role-admin: Middleware Kiểm Tra Quyền \(Authorization\)](#)
- **[Chương 7: MongoDB & Mongoose](#)**
 - [Bài 7.3: Giới Thiệu Mongoose - Kết Nối MongoDB](#)
 - [Bài 7.4: Thực Hành - CRUD Product Với Mongoose \(MVC\)](#)
- **[Chương 9: MongoDB Nâng Cao](#)**
 - [Bài 9.4: Query Operators - Toán Tử Truy Vấn MongoDB](#)
 - [Bài 9.th: Thực Hành - Explain Query, Index & Phân Trang](#)
- **[Chương 10: Xác Thực & Phiên Làm Việc](#)**
 - [Bài 10.2: Mã Hóa Mật Khẩu Với Bcrypt](#)
 - [Bài 10.3: Session & Middleware Xác Thực](#)

Chương 1: Giới Thiệu Node.js

Bài 1.4: Tạo HTTP Server Cơ Bản

Mục tiêu

- Tạo project Node.js với `npm init`
- Tạo HTTP server đơn giản
- Phân biệt 2 hệ thống module: **ES Modules** (`import`) và **CommonJS** (`require`)

Bước 1: Tạo thư mục project

Mở Terminal (hoặc Command Prompt trên Windows), chạy các lệnh sau:

```
mkdir my-node-project
cd my-node-project
```

Bước 2: Khởi tạo project Node.js

```
npm init -y
```

Lệnh này tạo file `package.json` - file cấu hình chính của project. Flag `-y` nghĩa là chấp nhận tất cả giá trị mặc định.

Bước 3: Bật chế độ ES Modules

Mở file `package.json` và thêm dòng `"type": "module"` :

```
{
  "name": "my-node-project",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "type": "module"
}
```

Tại sao cần `"type": "module"` ? Mặc định Node.js dùng CommonJS (`require`). Thêm dòng này để chuyển sang ES Modules (`import`) - cú pháp hiện đại hơn và là tiêu chuẩn của JavaScript.

Bước 4: Tạo HTTP Server với ES Modules

Tạo file `index.js` với nội dung sau:

```
import http from 'node:http'

const server = http.createServer((req, res) => {
  res.end('Hello world from Node.js!')
})

server.listen(3000, () => {
  console.log('Server is running ...')
})
```

Giải thích:

- `import http from 'node:http'` - Import module `http` có sẵn trong Node.js. Prefix `node:` cho biết đây là module built-in.
- `http.createServer()` - Tạo server, nhận callback với 2 tham số: `req` (request - yêu cầu từ client) và `res` (response - phản hồi từ server).
- `res.end('Hello world from Node.js!')` - Gửi nội dung text về cho client và kết thúc response.
- `server.listen(3000, callback)` - Server lắng nghe trên port 3000. Callback chạy khi server khởi động thành công.

Bước 5: Chạy server

```
node index.js
```

Kết quả mong đợi trên Terminal:

```
Server is running ...
```

Mở trình duyệt, truy cập địa chỉ: `http://localhost:3000`

Trình duyệt sẽ hiển thị: **Hello world from Node.js!**

Dừng server: Nhấn `Ctrl + C` trong Terminal.

Bước 6: So sánh với CommonJS (Tham khảo)

Tạo file `index.cjs` với nội dung sau:

```
const http = require('http')

const server = http.createServer((req, res) => {
  res.end('Hello world from Node.js!')
})

server.listen(3000, () => {
  console.log('Server is running ...')
})
```

```
console.log(__dirname)
console.log(__filename)
```

Chạy file này:

```
node index.cjs
```

Kết quả mong đợi:

```
/duong/dan/den/my-node-project
/duong/dan/den/my-node-project/index.cjs
Server is running ...
```

So sánh ES Modules vs CommonJS:

	ES Modules	CommonJS
Import	import http from 'node:http'	const http = require('http')
Extension	.js (khi có "type": "module")	.cjs hoặc .js (mặc định)
__dirname, __filename	Không có	Có sẵn
Tiêu chuẩn	JavaScript hiện đại	Truyền thống của Node.js

Lưu ý: File `.cjs` luôn được Node.js xử lý theo CommonJS, bất kể cấu hình `"type"` trong `package.json`.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
SyntaxError: Cannot use import statement outside a module	Thiếu "type": "module" trong package.json	Thêm "type": "module" vào package.json
EADDRINUSE: address already in use :::3000	Port 3000 đang bị chương trình khác chiếm	Tắt chương trình đang dùng port 3000 (có thể do chưa Ctrl+C server trước đó), hoặc đổi sang port khác (vd: 3001)
ReferenceError: __dirname is not defined	Dùng __dirname trong file ES Module	__dirname chỉ có trong CommonJS. Trong ES Module, sử dụng cách khác (sẽ học ở Chương 2)
ReferenceError: require is not defined in ES module scope	Dùng require() trong file ES Module	Đổi require() thành import

Chương 2: Các Module Built-in Của Node.js

Bài 2.2-os: Module OS - Thông Tin Hệ Điều Hành

Mục tiêu

- Sử dụng module `os` để lấy thông tin hệ điều hành
- Hiểu các hàm thường dùng: `platform()`, `arch()`, `hostname()`, `cpus()`, `totalmem()`

Bước 1: Tạo project

```
mkdir 2.2-os
cd 2.2-os
npm init -y
```

Mở `package.json`, thêm `"type": "module"`.

Bước 2: Viết code

Tạo file `index.js`:

```
import os from 'node:os'

console.log('Platform: ', os.platform());
console.log('Architecture: ', os.arch());
console.log('Hostname: ', os.hostname());
console.log('Hom Directory: ', os.homedir());

console.log("Number of CPU cores: ", os.cpus().length);

console.log("Total memory: ", os.totalmem() / 1024 / 1024 / 1024, " GB");

console.log("Nework details: ", os.networkInterfaces());
```

Giải thích các hàm:

Hàm	Ý nghĩa	Ví dụ kết quả
<code>os.platform()</code>	Hệ điều hành đang chạy	<code>darwin</code> (macOS), <code>win32</code> (Windows), <code>linux</code>
<code>os.arch()</code>	Kiến trúc CPU	<code>x64</code> , <code>arm64</code>
<code>os.hostname()</code>	Tên máy tính	<code>MacBook-Pro.local</code>
<code>os.homedir()</code>	Thư mục home của user	<code>/Users/ten</code> (macOS), <code>C:\Users\ten</code> (Windows)
<code>os.cpus().length</code>	Số lõi CPU	<code>8</code>
<code>os.totalmem()</code>	Tổng RAM (đơn vị byte)	Chia 3 lần cho 1024 để đổi sang GB
<code>os.networkInterfaces()</code>	Thông tin các card mạng	Object chứa IP, MAC address...

Bước 3: Chạy chương trình

```
node index.js
```

Kết quả mong đợi (giá trị khác nhau tùy máy):

```
Platform: darwin
Architecture: arm64
Hostname: MacBook-Pro.local
Hom Directory: /Users/ten
Number of CPU cores: 8
Total memory: 16 GB
Network details: { lo0: [...], en0: [...] }
```

Bài 2.2-path: Module Path - Xử Lý Đường Dẫn

Mục tiêu

- Sử dụng module `path` để thao tác với đường dẫn file
- Hiểu cách lấy `__filename` và `__dirname` trong ES Modules
- Các hàm: `dirname()`, `join()`, `extname()`, `resolve()`

Bước 1: Tạo project

```
mkdir 2.2-path
cd 2.2-path
npm init -y
```

Thêm `"type": "module"` vào `package.json`.

Tạo thư mục test và file test:

```
mkdir test-folder
touch test-folder/test.txt
```

Trên Windows dùng: `mkdir test-folder` rồi `echo. > test-folder\test.txt`

Bước 2: Viết code

Tạo file `index.js`:

```
import { fileURLToPath } from 'url';
import path from 'path';

const filePath = fileURLToPath(import.meta.url);
console.log(filePath);
const dirPath = path.dirname(filePath)
console.log(dirPath)
```

```
const filePath_2 = path.join(dirPath, "index.js");
console.log(filePath_2);
console.log(path.extname(filePath_2));

const testFolderPath = 'test-folder';
const testFilePath = path.join(testFolderPath, "test.txt");
const absolutePath = path.resolve(testFilePath);
console.log(absolutePath)
```

Giải thích:

- **import.meta.url** - Trong ES Module, đây là cách lấy URL của file hiện tại (dạng `file:///duong/dan/index.js`).
- **fileURLToPath(import.meta.url)** - Chuyển URL thành đường dẫn file bình thường, thay thế cho `__filename` (chỉ có trong CommonJS).
- **path.dirname(filePath)** - Lấy đường dẫn thư mục chứa file, thay thế cho `__dirname`.
- **path.join(dirPath, "index.js")** - Nối các phần đường dẫn lại với nhau. Tự động thêm `/` hoặc `\` tùy hệ điều hành.
- **path.extname(filePath_2)** - Lấy phần mở rộng của file (vd: `.js`, `.txt`).
- **path.resolve(testFilePath)** - Chuyển đường dẫn tương đối thành đường dẫn tuyệt đối.

Bước 3: Chạy chương trình

```
node index.js
```

Kết quả mong đợi:

```
/duong/dan/2.2-path/index.js
/duong/dan/2.2-path
/duong/dan/2.2-path/index.js
.js
/duong/dan/2.2-path/test-folder/test.txt
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
ReferenceError: __dirname is not defined	Dùng <code>__dirname</code> trong ES Module	Dùng <code>path.dirname(fileURLToPath(import.meta.url))</code> thay thế
ReferenceError: __filename is not defined	Dùng <code>__filename</code> trong ES Module	Dùng <code>fileURLToPath(import.meta.url)</code> thay thế

Bài 2.2-fs: Module File System - Đọc/Ghi File

Mục tiêu

- Sử dụng module `fs` để đọc file
- Hiểu sự khác nhau giữa: đồng bộ (sync), bất đồng bộ callback, Promise, và Promise.all

- Hiểu khái niệm **blocking** vs **non-blocking**

Bước 1: Tạo project và dữ liệu

```
mkdir 2.2-fs
cd 2.2-fs
npm init -y
```

Thêm `"type": "module"` vào `package.json` .

Tạo 3 file text để test:

```
echo "DAY LA FILE 1." > file1.txt
echo "DAY LA FILE 2" > file2.txt
echo "DAY LA FILE 3." > file3.txt
```

Bước 2: Đọc file đồng bộ (Synchronous)

Tạo file `readFileSync.js` :

```
import fs from 'fs'

console.log('Đang đọc file đồng bộ ...');

const data = fs.readFileSync('file1.txt', 'utf-8');
console.log('Data: ', data);

console.log('=====');
```

Chạy:

```
node readFileSync.js
```

Kết quả:

```
Đang đọc file đồng bộ ...
Data:  DAY LA FILE 1.
=====
```

Giải thích: `readFileSync` là hàm **đồng bộ (blocking)** - chương trình dừng lại chờ đọc file xong rồi mới chạy dòng tiếp theo. Vì vậy kết quả luôn theo thứ tự: log -> data -> dấu `=` .

Bước 3: Đọc file bất đồng bộ với Callback

Tạo file `readFile.js` :

```
import fs from 'fs'

try {
  console.log('Đang đọc file bất đồng bộ ...');
```



```
fs.readFile('file1.txt', 'utf-8', (err, data) => {
    console.log('Data: ', data);
})
);
} catch (error) {
    console.log('Loi doc file: ', error.message)
}

console.log('=====');
```

Chạy:

```
node readFile.js
```

Kết quả:

```
Đang đọc file bất đồng bộ ...
=====
Data:  DAY LA FILE 1.
```

Giải thích: `readFile` là hàm **bất đồng bộ (non-blocking)** - chương trình KHÔNG chờ đọc file xong mà chạy tiếp dòng `console.log('=====')` ngay. Khi đọc file xong, callback được gọi và in ra data. Vì vậy dấu `=` xuất hiện TRƯỚC data.

Đây là điểm khác biệt quan trọng nhất: Đồng bộ chờ xong mới chạy tiếp, bất đồng bộ chạy tiếp ngay không chờ.

Bước 4: Đọc file với `async/await` (`fs/promises`)

Tạo file `readPromise.js` :

```
import fs from 'node:fs/promises'

async function docFile(fileName) {
    const data = await fs.readFile(fileName, 'utf-8');
    console.log(`Data of ${fileName}: `, data);
}

(async () => {
    await docFile('file1.txt');
})();
```

Chạy:

```
node readPromise.js
```

Kết quả:

```
Data of file1.txt:  DAY LA FILE 1.
```

Giải thích: `fs/promises` trả về Promise thay vì dùng callback. Kết hợp với `async/await` giúp code gọn gàng và dễ đọc hơn. `await` sẽ chờ Promise hoàn thành, nhưng **không block** toàn bộ chương trình - chỉ tạm dừng trong hàm `async` đó.

Bước 5: Đọc nhiều file tuần tự với Promise chaining

Tạo file `promise.js` :

```
import fs from 'fs/promises'

console.log('Đang đọc file bất đồng bộ ...');

fs.readFile('file1.txt', 'utf-8')
  .then((data) => {
    console.log('Data: ', data);
    return fs.readFile('./file2.txt', 'utf-8');
  })
  .then((data2) => {
    console.log('Data file 2: ', data2);
    return fs.readFile('./file3.txt', 'utf-8');
  })
  .then((data3) => {
    console.log('Data file 3: ', data3);
  })
  .catch((error) => {
    console.error('Loi: ', error.message);
  })
  .finally(() => {
    console.log('=====');
  })
```

Chạy:

```
node promise.js
```

Kết quả:

```
Đang đọc file bất đồng bộ ...
Data:  DAY LA FILE 1.
Data file 2:  DAY LA FILE 2
Data file 3:  DAY LA FILE 3.
=====
```

Giải thích: Đọc file1 xong -> đọc file2 -> đọc file3, theo thứ tự nối tiếp nhau (tuần tự). `.catch()` bắt lỗi nếu bất kỳ bước nào thất bại. `.finally()` luôn chạy dù thành công hay lỗi.

Bước 6: Đọc nhiều file song song với Promise.all

Tạo file `readPromiseAll.js` :

```
import fs from 'node:fs/promises'

try {
  console.log('Đang đọc file với fs/promises ...');

  const startTime = performance.now();
  const [data, data2, data3] = await Promise.all([
    fs.readFile('./file1.txt', 'utf-8'),
    fs.readFile('./file2.txt', 'utf-8'),
    fs.readFile('./file3.txt', 'utf-8')
  ])

  console.log('Data: ', data);
  console.log('Data file 2: ', data2);
  console.log('Data file 3: ', data3);
  const endTime = performance.now();
  const duration = (endTime - startTime).toFixed(3);
  console.log('Execution time: ', duration);

} catch (error) {
  console.log('Lỗi đọc file: ', error.message);
}

console.log('=====');
```

Chạy:

```
node readPromiseAll.js
```

Kết quả:

```
Đang đọc file với fs/promises ...
Data: DAY LA FILE 1.
Data file 2: DAY LA FILE 2
Data file 3: DAY LA FILE 3.
Execution time: 1.234
=====
```

Giải thích: `Promise.all()` chạy tất cả 3 thao tác đọc file **cùng lúc** (song song), không chờ file trước đọc xong mới đọc file sau. Kết quả trả về là mảng, dùng **destructuring** `[data, data2, data3]` để lấy từng giá trị. Nhanh hơn đáng kể so với đọc tuần tự khi có nhiều file.

Tóm tắt tiến hóa cách đọc file:

Cách	Ưu điểm	Nhược điểm
<code>readFileSync</code>	Đơn giản, dễ hiểu	Blocking - chương trình đứng chờ
<code>readFile (callback)</code>	Non-blocking	Code lồng nhau phức tạp (callback hell)
<code>fs/promises + async/await</code>	Gọn gàng, dễ đọc	Cần hiểu Promise

`Promise.all`

Song song, nhanh nhất

Nếu 1 file lỗi thì tất cả đều lỗi

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
ENOENT: no such file or directory, open 'file1.txt'	File không tồn tại hoặc sai tên	Kiểm tra file tồn tại và đúng tên. Chú ý chạy lệnh node từ đúng thư mục chứa file
SyntaxError: Cannot use import statement outside a module	Thiếu "type": "module"	Thêm "type": "module" vào package.json
The "data" argument must be of type string...	Thiếu tham số 'utf-8' khi đọc file	Thêm 'utf-8' làm tham số thứ 2: fs.readFile('file.txt', 'utf-8', ...)
Warning: To load an ES module, set "type": "module"...	Dùng await ở top-level nhưng file không phải ES Module	Thêm "type": "module" vào package.json

Bài 2.th-01: Thực Hành Tổng Hợp - Đọc JSON & Ghi Báo Cáo

Mục tiêu

- Áp dụng `Promise.all` để đọc nhiều file JSON cùng lúc
- Sử dụng `JSON.parse()` để chuyển chuỗi JSON thành object JavaScript
- Sử dụng `fs.writeFile()` để ghi kết quả ra file
- Kết hợp các kiến thức đã học trong chương 2

Bước 1: Tạo project

```
mkdir 2.th-01
cd 2.th-01
npm init -y
```

Thêm `"type": "module"` vào `package.json`.

Bước 2: Tạo dữ liệu - các file JSON danh sách sinh viên

Tạo file `lopA.json`:

```
{
  "students": [
    {
      "id": 1,
      "name": "tuan"
    },
    {
      "id": 2,
      "name": "vu"
    },
  ],
}
```

```
{
  "id": 3,
  "name": "nguyễn"
},
{
  "id": 4,
  "name": "trần"
}
]
```

Tạo file `lopB.json` :

```
{
  "students": [
    {
      "id": 1,
      "name": "tuan"
    },
    {
      "id": 2,
      "name": "vu"
    },
    {
      "id": 3,
      "name": "nguyễn"
    }
  ]
}
```

Tạo file `lopC.json` :

```
{
  "students": [
    {
      "id": 1,
      "name": "tuan"
    },
    {
      "id": 2,
      "name": "vu"
    }
  ]
}
```

Bước 3: Viết code xử lý

Tạo file `index.js` :

```
import fs from 'node:fs/promises'

console.log("Đọc file ...");

try {
  const [dulieuLopA, dulieuLopB, dulieuLopC] = await Promise.all([
    fs.readFile('./lopA.json', 'utf-8'),
    fs.readFile('./lopB.json', 'utf-8'),
    fs.readFile('./lopC.json', 'utf-8'),
  ]);

  const sinhvienLopA = JSON.parse(dulieuLopA);
  const sinhvienLopB = JSON.parse(dulieuLopB);
  const sinhvienLopC = JSON.parse(dulieuLopC);

  const ketqua = `Có tất cả ${sinhvienLopA.students.length +
    sinhvienLopB.students.length + sinhvienLopC.students.length} sinh viên`
  await fs.writeFile("baocao.txt", ketqua, "utf-8");
  console.log("Đã lưu kết quả vào file baocao.txt");
} catch (error) {
  console.log("Lỗi: ", error.message);
}
```

Giải thích:

1. **Đọc 3 file JSON song song** bằng `Promise.all` - cả 3 file được đọc cùng lúc, kết quả trả về là mảng 3 chuỗi.
2. **`JSON.parse()`** - Chuyển chuỗi JSON (text) thành object JavaScript. Sau bước này có thể truy cập `sinhvienLopA.students` như một mảng bình thường.
3. **Tính tổng** - Dùng `.length` để đếm số phần tử trong mảng `students` của mỗi lớp, rồi cộng lại: $4 + 3 + 2 = 9$.
4. **Template literal** - Dùng backtick ``` và `${}` để chèn biểu thức JavaScript vào chuỗi.
5. **`fs.writeFile()`** - Ghi nội dung vào file `baocao.txt`. Nếu file chưa tồn tại sẽ tự tạo mới, nếu đã tồn tại sẽ ghi đè.

Bước 4: Chạy chương trình

```
node index.js
```

Kết quả trên Terminal:

```
Đọc file ...
Đã lưu kết quả vào file baocao.txt
```

Kiểm tra file `baocao.txt` được tạo ra:

```
cat baocao.txt
```

Nội dung file:

Có tất cả 9 sinh viên

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
SyntaxError: Unexpected token } in JSON at position...	File JSON sai cú pháp (thừa dấu phẩy, thiếu dấu ngoặc...)	Kiểm tra lại file JSON. Mỗi key và value chuỗi phải dùng dấu ngoặc kép ". Không được có dấu phẩy sau phần tử cuối cùng
TypeError: Cannot read properties of undefined (reading 'length')	Tên property sai (vd: student thay vì students)	Kiểm tra tên property trong file JSON và trong code phải khớp nhau
ENOENT: no such file or directory, open './lopA.json'	File JSON không tồn tại hoặc sai đường dẫn	Đảm bảo các file .json nằm cùng thư mục với index.js
File baocao.txt bị ghi đè mỗi lần chạy	writeFile mặc định ghi đè toàn bộ	Đây là hành vi bình thường. Nếu muốn ghi thêm (append), dùng fs.appendFile() thay vì fs.writeFile()

Chương 3: Express.js Cơ Bản

Bài 3.1: Node Modules & NPM

Mục tiêu

- Hiểu cách quản lý package với NPM
- Cài đặt và sử dụng package bên ngoài (`express` , `nodemon`)
- Sử dụng `nodemon` để tự động restart server khi thay đổi code

Bước 1: Tạo project

```
mkdir 3.1.node-modules
cd 3.1.node-modules
npm init -y
```

Thêm `"type": "module"` vào `package.json` .

Bước 2: Cài đặt Express và Nodemon

```
npm install express
npm install nodemon --save-dev
```

Giải thích:

- `npm install express` - Cài Express vào `dependencies` (cần cho production).
- `npm install nodemon --save-dev` - Cài Nodemon vào `devDependencies` (chỉ cần khi phát triển). Nodemon tự động restart server mỗi khi bạn lưu file, không cần `Ctrl+C` rồi `node index.js` lại.

Sau khi cài, thư mục `node_modules/` và file `package-lock.json` sẽ được tạo tự động.

Bước 3: Thêm script chạy với Nodemon

Mở `package.json` , thêm script `"dev"` :

```
{
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "dev": "nodemon index.js"
  }
}
```

Bước 4: Viết code

Tạo file `index.js` :

```
import http from 'node:http'
```



```
const server = http.createServer((req, res) => {
  res.end('Hello world!');
})

server.listen(3000, () => {
  console.log('Server is running ...');
})
```

Bước 5: Tạo file .gitignore

Tạo file `.gitignore` :

```
node_modules
```

Tại sao cần `.gitignore` ? Thư mục `node_modules/` rất lớn (hàng ngàn file). Không nên đưa lên Git. Khi cần cài lại, chỉ cần chạy `npm install` - NPM sẽ đọc `package.json` và tải lại tất cả.

Bước 6: Chạy server

Cách 1 - Chạy thường:

```
node index.js
```

Cách 2 - Chạy với Nodemon (khuyến dùng khi phát triển):

```
npm run dev
```

Khi dùng Nodemon, mỗi khi bạn sửa và lưu file `index.js` , server sẽ tự restart.

Kết quả:

```
Server is running ...
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Error: Cannot find module 'express'	Chưa cài express	Chạy <code>npm install express</code>
'nodemon' is not recognized as a command	Chưa cài nodemon hoặc chưa dùng script	Chạy <code>npm install nodemon --save-dev</code> rồi dùng <code>npm run dev</code> thay vì <code>nodemon index.js</code> trực tiếp
Thư mục <code>node_modules</code> quá lớn	Đây là bình thường	Thêm <code>node_modules</code> vào <code>.gitignore</code> , không đưa lên Git

Bài 3.3: Express, Morgan & Các Loại Tham Số Request

Mục tiêu

- Tạo ứng dụng Express.js đầu tiên

- Sử dụng middleware Morgan để ghi log request
- Phân biệt 3 loại tham số: **Query Params**, **Route Params**, **Request Body**
- Gửi request POST với custom header

Bước 1: Tạo project và cài đặt

```
mkdir 3.3-morgan
cd 3.3-morgan
npm init -y
npm install express morgan
```

Thêm `"type": "module"` vào `package.json` .

Bước 2: Viết code

Tạo file `index.js` :

```
import express from 'express';
import morgan from 'morgan';

const app = express();
app.use(express.json())
app.use(morgan('common'))

// params

// 1. query params
// http://localhost:3000?search=test
app.get('/', (req, res) => {
  res.end('Hello ' + req.query.search);
})

// 2. route params
// http://localhost:3000/test
app.get('/:search', (req, res) => {
  res.end('Hello ' + req.params.search + ' with name: ' + req.query.search);
})

// body
app.post('/', (req, res) => {
  const header_info = req.headers['my-header']
  console.log('my-header: ', header_info);

  res.json(req.body.university)
})

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Giải thích:

- `express()` - Tạo ứng dụng Express.
- `app.use(express.json())` - Middleware giúp Express đọc được dữ liệu JSON từ body request.
- `app.use(morgan('common'))` - Middleware ghi log mỗi request vào Terminal (IP, thời gian, method, URL, status code).
- **Query Params** (`req.query`) - Tham số trên URL sau dấu `?` . VD: `?search=test` -> `req.query.search = "test"` .
- **Route Params** (`req.params`) - Tham số nằm trong đường dẫn URL. VD: `/:search` khi truy cập `/hello` -> `req.params.search = "hello"` .
- **Request Body** (`req.body`) - Dữ liệu gửi kèm trong body (thường dùng với POST/PUT).
- **Request Headers** (`req.headers`) - Thông tin header của request.

Bước 3: Chạy và test

```
node index.js
```

Test 1 - Query Params: Mở trình duyệt:

```
http://localhost:3000?search=nodejs
```

Kết quả: Hello nodejs

Test 2 - Route Params: Mở trình duyệt:

```
http://localhost:3000/express?search=framework
```

Kết quả: Hello express with name: framework

Test 3 - POST request: Dùng công cụ test API (Postman, Thunder Client trong VS Code, hoặc curl):

```
curl -X POST http://localhost:3000 \
  -H "Content-Type: application/json" \
  -H "my-header: xin-chao" \
  -d '{"university": "Dong A"}'
```

Kết quả trả về: "Dong A"

Terminal hiển thị log Morgan:

```
:::1 - - [12/May/2025:10:30:00 +0000] "GET /?search=nodejs HTTP/1.1" 200 12
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
<code>req.body</code> là undefined	Thiếu <code>app.use(express.json())</code>	Thêm <code>app.use(express.json())</code> trước các route
<code>req.query.search</code> là undefined	URL thiếu query param	Đảm bảo URL có dạng <code>?search=giaTri</code>
Cannot GET /favicon.ico	Trình duyệt tự request icon	Bỏ qua, đây là hành vi bình thường của trình duyệt

Bài 3.th: Thực Hành - Xây Dựng RESTful API CRUD Sinh Viên

Mục tiêu

- Xây dựng API RESTful hoàn chỉnh với 5 endpoint CRUD
- Sử dụng đúng HTTP method cho từng thao tác
- Kết hợp route params và request body

Bước 1: Tạo project và cài đặt

```
mkdir 3.th
cd 3.th
npm init -y
npm install express morgan
npm install nodemon --save-dev
```

Thêm "type": "module" và script "dev" vào package.json :

```
{
  "type": "module",
  "scripts": {
    "dev": "nodemon index.js"
  }
}
```

Bước 2: Viết code

Tạo file index.js :

```
import express from 'express';
import morgan from 'morgan';

const app = express();
app.use(express.json())

app.use(morgan('dev'))

app.get('/sinhvien/', (req, res) => {
  res.send('Danh sách sinh viên là ...')
})

app.post('/sinhvien/', (req, res) => {
  res.json(req.body);
})

app.get('/sinhvien/:id', (req, res) => {
  res.send('Thông tin sinh viên có mã ' + req.params.id)
})

app.put('/sinhvien/:id', (req, res) => {
```

```

    res.send('Đã sửa thông tin sinh viên có mã ' + req.params.id + ' với chi tiết: '
+ JSON.stringify(req.body))
  })

app.delete('/sinhvien/:id', (req, res) => {
  // kiểm tra có sinh viên :id không
  // nếu có thì xoá
  res.send('Đã xoá sinh viên ' + req.params.id)
})

app.listen(3000, () => {
  console.log('server is running ...');
})

```

Giải thích:

Đây là mô hình **RESTful API** - sử dụng HTTP method phù hợp với từng thao tác:

HTTP Method	Endpoint	Thao tác	Ý nghĩa
GET	/sinhvien/	Read all	Lấy danh sách tất cả sinh viên
POST	/sinhvien/	Create	Tạo sinh viên mới
GET	/sinhvien/:id	Read one	Lấy thông tin 1 sinh viên theo mã
PUT	/sinhvien/:id	Update	Cập nhật thông tin sinh viên
DELETE	/sinhvien/:id	Delete	Xoá sinh viên

- `morgan('dev')` - Format log ngắn gọn, có màu sắc: `GET /sinhvien/ 200 3.456 ms`
- `res.send()` - Gửi text response.
- `res.json()` - Gửi JSON response.
- `JSON.stringify(req.body)` - Chuyển object thành chuỗi JSON để hiển thị.

Bước 3: Chạy và test

```
npm run dev
```

Test bằng curl hoặc Postman:

```

# Lấy danh sách sinh viên
curl http://localhost:3000/sinhvien/

# Tạo sinh viên mới
curl -X POST http://localhost:3000/sinhvien/ \
  -H "Content-Type: application/json" \
  -d '{"name": "Nguyen Van A", "lop": "20A"}'

# Xem sinh viên có mã 1
curl http://localhost:3000/sinhvien/1

```

```
# Sửa sinh viên có mã 1
curl -X PUT http://localhost:3000/sinhvien/1 \
  -H "Content-Type: application/json" \
  -d '{"name": "Nguyen Van B"}'

# Xoá sinh viên có mã 1
curl -X DELETE http://localhost:3000/sinhvien/1
```

Terminal hiển thị log Morgan (format `dev`):

```
GET /sinhvien/ 200 1.234 ms - 30
POST /sinhvien/ 200 0.567 ms - 42
GET /sinhvien/1 200 0.345 ms - 35
PUT /sinhvien/1 200 0.432 ms - 58
DELETE /sinhvien/1 200 0.234 ms - 20
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Cannot GET /sinhvien (không có / cuối)	Đường dẫn không khớp	Thử cả /sinhvien và /sinhvien/, hoặc bỏ / cuối trong route
POST trả về {} rỗng	Thiếu Content-Type: application/json trong header	Thêm header -H "Content-Type: application/json" khi gửi request
req.params.id luôn là string	Route params luôn trả về string	Dùng Number(req.params.id) hoặc parseInt(req.params.id) nếu cần so sánh số

Chương 4: Kiến Trúc MVC

Bài 4.0: So Sánh HTTP Thuần vs Express

Mục tiêu

- Hiểu tại sao cần Express.js
- So sánh cách tạo server và xử lý route giữa module `http` thuần và Express

Bước 1: Tạo project

```
mkdir 4.0.no-express
cd 4.0.no-express
npm init -y
npm install express
```

Thêm `"type": "module"` vào `package.json` .

Bước 2: Tạo server KHÔNG dùng Express

Tạo file `index.js` :

```
import http from 'http'

const server = http.createServer((req, res) => {
  if (req.url === '/sinhvien') {
    res.end('Hello from Sinhvien!');
  }
  else if (req.url === '/') {
    res.end('Hello world!');
  }
})

server.listen(3000, () => {
  console.log('Server is running ...');
})
```

Bước 3: Tạo server CÓ dùng Express

Tạo file `index-express.js` :

```
import express from 'express'

const app = express();

app.get('/', (req, res) => {
  res.send('Hello world from express!')
})
app.get('/sinhvien', (req, res) => {
  res.send('Hello from Sinh vien Express!')
```

```
})  
  
app.listen(3000, () => {  
  console.log('Server is running with express ...');  
})
```

Bước 4: Chạy và so sánh

```
# Chạy bản HTTP thuần  
node index.js  
  
# Hoặc chạy bản Express (dùng server trước bằng Ctrl+C)  
node index-express.js
```

Truy cập `http://localhost:3000` và `http://localhost:3000/sinhvien` trên trình duyệt.

So sánh:

	HTTP thuần (http)	Express
Routing	Dùng if/else if kiểm tra req.url	Dùng app.get(), app.post()... rõ ràng
Response	res.end()	res.send(), res.json() - nhiều tiện ích
Middleware	Phải tự viết	Có sẵn hệ thống middleware
Khi nhiều route	Code rất dài, khó quản lý	Mỗi route 1 block, dễ đọc

Express giúp code ngắn gọn, dễ đọc và dễ mở rộng hơn rất nhiều so với dùng module `http` trực tiếp.

Bài 4.4-mvc: Giới Thiệu Kiến Trúc MVC

Mục tiêu

- Hiểu mô hình **MVC (Model - View - Controller)**
- Tách code thành các tầng: Model, Controller, Route
- Sử dụng **Express Router** để quản lý route

Kiến thức MVC

Request → Route → Controller → Model → Response

Tầng	Vai trò	Ví dụ
Model	Quản lý dữ liệu	Mảng users, kết nối database
Controller	Xử lý logic nghiệp vụ	Lấy dữ liệu, tính toán, trả response
Route	Định nghĩa endpoint & HTTP method	GET /users, POST /users
View	Giao diện (sẽ học ở Chương 5)	Template HTML

Bước 1: Tạo project và cấu trúc thư mục

```
mkdir 4.4-mvc
cd 4.4-mvc
npm init -y
npm install express
npm install nodemon --save-dev
```

Thêm "type": "module" và script "dev": "nodemon app.js" vào package.json .

Tạo cấu trúc thư mục:

```
mkdir models controllers routes views
```

Cấu trúc project:

```
4.4-mvc/
├─ app.js
├─ models/
│   └─ user.models.js
├─ controllers/
│   └─ user.controller.js
├─ routes/
│   └─ user.route.js
└─ views/    (trống – sẽ dùng ở chương sau)
```

Bước 2: Tạo Model

Tạo file models/user.models.js :

```
let users = [
  {
    id: 1,
    name: "tuan"
  },
  {
    id: 2,
    name: "vu"
  }
]

export default users
```

Giải thích: Model chứa dữ liệu. Hiện tại dùng mảng trong bộ nhớ, sau này sẽ thay bằng database.

Bước 3: Tạo Controller

Tạo file controllers/user.controller.js :

```
import users from '../models/user.models.js'

export const getAll = (req, res) => {
  res.json(users)
}

export const create = (req, res) => {
  const user = {
    id: users.length + 1,
    name: req.body.name
  }
  users.push(user)
  res.json(user)
}
```

Giải thích: Controller chứa logic xử lý. Mỗi hàm nhận `req`, `res` và thực hiện một thao tác cụ thể. `export const` cho phép export từng hàm riêng lẻ (named export).

Bước 4: Tạo Route

Tạo file `routes/user.route.js` :

```
import { Router } from "express";
import { getAll, create } from "../controllers/user.controller.js";

const router = Router()

router.get('', getAll)
router.post('', create)

export default router
```

Giải thích: `Router()` tạo một mini-router riêng. Các route ở đây dùng đường dẫn `''` (rỗng) vì prefix `/users` sẽ được gắn trong `app.js`.

Bước 5: Tạo file chính app.js

Tạo file `app.js` :

```
import express from 'express'
import userRouter from './routes/user.route.js'

const app = express()
app.use(express.json())

app.use('/users', userRouter)

app.listen(3000, (req, res) => {
  console.log('Server is running ...');
})
```

Giải thích: `app.use('/users', userRouter)` - Gắn toàn bộ `userRouter` vào prefix `/users` . Nghĩa là:

- `router.get('')` -> GET `/users`
- `router.post('')` -> POST `/users`

Bước 6: Chạy và test

```
npm run dev
```

Test lấy danh sách users:

```
curl http://localhost:3000/users
```

Kết quả:

```
[{"id":1,"name":"tuan"}, {"id":2,"name":"vu"}]
```

Test tạo user mới:

```
curl -X POST http://localhost:3000/users \
-H "Content-Type: application/json" \
-d '{"name": "nguyen"}
```

Kết quả:

```
{"id":3,"name":"nguyen"}
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
ERR_MODULE_NOT_FOUND khi import	Thiếu đuôi <code>.js</code> trong đường dẫn import	ES Module yêu cầu ghi đầy đủ: <code> './routes/user.route.js'</code> (không được bỏ <code>.js</code>)
TypeError: Router is not a function	Import sai cú pháp	Dùng <code>import { Router } from "express"</code> (destructuring)
Cannot GET /users/ (có / cuối)	Route định nghĩa không có / cuối	Thử bỏ / cuối trong URL hoặc thêm / trong route

Bài 4.4.thuchanh: Thực Hành - API Users Đơn Giản (Chứa MVC)

Mục tiêu

- Viết API quản lý users đơn giản trong 1 file
- Hiểu tại sao cần tách ra MVC khi code lớn dần

Bước 1: Tạo project

```
mkdir 4.4.thuchanh
cd 4.4.thuchanh
npm init -y
npm install express
npm install nodemon --save-dev
```

Thêm `"type": "module"` và script `"dev"` vào `package.json` .

Bước 2: Viết code (tất cả trong 1 file)

Tạo file `index.js` :

```
import express from 'express'

const app = express()
app.use(express.json())

let users = [
  {
    id: 1,
    name: "tuan"
  },
  {
    id: 2,
    name: "vu"
  }
]

app.get('/users', (req, res) => {
  res.json(users)
})

app.post('/users', (req, res) => {
  const user = {
    id: users.length + 1,
    name: req.body.name
  }
  users.push(user)
  res.json(user)
})

app.listen(3000, () => {
  console.log('Server is running');
})
```

Bước 3: Chạy và test

```
npm run dev
```

Test giống bài 4.4-mvc ở trên. Kết quả giống nhau, nhưng **tất cả code nằm trong 1 file**.

Nhận xét: Bài này làm được nhưng khi project lớn (nhiều resource: users, products, orders...), file sẽ rất dài và khó quản lý. Đó là lý do cần tách ra theo MVC như bài 4.4-mvc.

Bài 4.4.thuchanh-2: Thực Hành - CRUD Đầy Đủ Với MVC (Nhiều Resource)

Mục tiêu

- Áp dụng MVC cho nhiều resource (customers & products)
- Viết đầy đủ 5 thao tác CRUD: getAll, getOne, create, update, delete
- Xử lý lỗi với HTTP status code phù hợp (404, 204)

Bước 1: Tạo project và cấu trúc

```
mkdir 4.4.thuchanh-2
cd 4.4.thuchanh-2
npm init -y
npm install express
npm install nodemon --save-dev
```

Thêm "type": "module" và script "dev": "nodemon index.js" vào package.json .

```
mkdir models controllers routes
```

Cấu trúc project:

```
4.4.thuchanh-2/
├─ index.js
├─ models/
│   └─ customer.model.js
│   └─ product.model.js
├─ controllers/
│   └─ customer.controller.js
│   └─ product.controller.js
└─ routes/
    └─ customer.route.js
    └─ product.route.js
```

Bước 2: Tạo Models

Tạo file models/customer.model.js :

```
let customers = [
  {
    id: 1,
    name: "tuan"
  },
  {
    id: 2,
    name: "vu"
  }
]
```

```
    },  
    {  
      id: 3,  
      name: "nguyen"  
    }  
  ]  
  
  export default customers
```

Tạo file `models/product.model.js` :

```
let products = [  
  {  
    id: 1,  
    name: "Laptop",  
    price: 1000  
  },  
  {  
    id: 2,  
    name: "Phone",  
    price: 200  
  }  
]  
  
export default products
```

Bước 3: Tạo Controllers (CRUD đầy đủ)

Tạo file `controllers/customer.controller.js` :

```
import customers from "../models/customer.model.js"  
  
export const getAll = (req, res) => {  
  res.json(customers)  
}  
  
export const create = (req, res) => {  
  const newCustomer = {  
    id: customers.length + 1,  
    ...req.body  
  }  
  customers.push(newCustomer)  
  res.json(newCustomer)  
}  
  
export const getOne = (req, res) => {  
  const customer = customers.find(c => c.id === Number(req.params.id))  
  if (!customer) {  
    return res.status(404).json({ error: 'Not found' })  
  }  
  res.json(customer)  
}
```

```

}

export const update = (req, res) => {
  const index = customers.findIndex(c => c.id === Number(req.params.id))
  if (index === -1) {
    return res.status(404).json({ error: 'Not found' })
  }
  customers[index] = {
    ...customers[index],
    ...req.body
  }
  res.json(customers[index])
}

export const remove = (req, res) => {
  const index = customers.findIndex(c => c.id === Number(req.params.id))
  if (index === -1) {
    return res.status(404).json({ error: 'Not found' })
  }
  customers.splice(index, 1)
  res.status(204).end()
}

```

Giải thích các kỹ thuật quan trọng:

- **...req.body** (Spread operator) - Sao chép tất cả property từ body vào object mới, không cần liệt kê từng field.
- **Number(req.params.id)** - Chuyển string thành number vì `req.params` luôn trả về string.
- **.find()** - Tìm phần tử đầu tiên khớp điều kiện, trả về `undefined` nếu không tìm thấy.
- **.findIndex()** - Tìm vị trí (index) của phần tử, trả về `-1` nếu không tìm thấy.
- **.splice(index, 1)** - Xóa 1 phần tử tại vị trí `index`.
- **res.status(404)** - Trả về HTTP status 404 (Not Found).
- **res.status(204).end()** - Trả về HTTP status 204 (No Content) - thành công nhưng không có nội dung trả về (thường dùng cho DELETE).

Tạo file `controllers/product.controller.js` (tương tự, thay `customers` bằng `products`):

```

import products from "../models/product.model.js";

export const getAll = (req, res) => {
  res.json(products)
}

export const create = (req, res) => {
  const newProduct = {
    id: products.length + 1,
    ...req.body
  }
  products.push(newProduct)
  res.json(newProduct)
}

```

```

export const getOne = (req, res) => {
  const product = products.find(p => p.id === Number(req.params.id))
  if (!product) {
    return res.status(404).json({ error: 'Not found' })
  }
  res.json(product)
}

export const update = (req, res) => {
  const index = products.findIndex(p => p.id === Number(req.params.id))
  if (index === -1) {
    return res.status(404).json({ error: 'Not found' })
  }
  products[index] = {
    ...products[index],
    ...req.body
  }
  res.json(products[index])
}

export const remove = (req, res) => {
  const index = products.findIndex(p => p.id === Number(req.params.id))
  if (index === -1) {
    return res.status(404).json({ error: 'Not found' })
  }
  products.splice(index, 1)
  res.status(204).end()
}

```

Bước 4: Tạo Routes

Tạo file `routes/customer.route.js` :

```

import { Router } from "express";
import { create, getAll, getOne, remove, update } from
"../controllers/customer.controller.js";

const router = new Router();

router.get('/', getAll)
router.get('/:id', getOne)
router.put('/:id', update)
router.delete('/:id', remove)
router.post('/', create)

export default router;

```

Tạo file `routes/product.route.js` :

```

import { Router } from "express";
import { create, getAll, getOne, remove, update } from

```



```
 "../controllers/product.controller.js";

const router = Router()

router.get('/', getAll)
router.get('/:id', getOne)
router.put('/:id', update)
router.delete('/:id', remove)
router.post('/', create)

export default router
```

Bước 5: Tạo file chính

Tạo file `index.js` :

```
import express from 'express'
import productRouter from './routes/product.route.js'
import customerRouter from './routes/customer.route.js'

const app = express();
app.use(express.json())

app.use('/products', productRouter)
app.use('/customers', customerRouter)

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Bước 6: Chạy và test

```
npm run dev
```

Test Customer API:

```
# Lấy tất cả customers
curl http://localhost:3000/customers

# Lấy customer có id = 1
curl http://localhost:3000/customers/1

# Tạo customer mới
curl -X POST http://localhost:3000/customers \
  -H "Content-Type: application/json" \
  -d '{"name": "tran"}'

# Cập nhật customer có id = 1
curl -X PUT http://localhost:3000/customers/1 \
  -H "Content-Type: application/json" \
```

```
-d '{"name": "tuan updated"}'
```

Xóa customer có id = 2

```
curl -X DELETE http://localhost:3000/customers/2
```

Test Product API:

```
# Lấy tất cả products
curl http://localhost:3000/products

# Tạo product mới
curl -X POST http://localhost:3000/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Tablet", "price": 500}'

# Lấy product không tồn tại
curl http://localhost:3000/products/99
```

Kết quả khi product không tồn tại (status 404):

```
{"error": "Not found"}
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
ERR_MODULE_NOT_FOUND	Thiếu .js trong import path	Luôn ghi đầy đủ đuôi .js: '../models/customer.model.js'
GET /customers/abc trả về 404	Number("abc") = NaN, không khớp id nào	Đây là đúng hành vi. Có thể thêm validation nếu muốn thông báo lỗi rõ hơn
DELETE trả về rỗng	Status 204 nghĩa là No Content	Đúng hành vi. Kiểm tra status code = 204 là thành công
Dữ liệu mất khi restart server	Dữ liệu lưu trong bộ nhớ (mảng)	Đây là bình thường. Sẽ học lưu vào database ở các chương sau

Chương 5: Template Engine - EJS

Bài 5.2: Ứng Dụng Web Với EJS

Mục tiêu

- Sử dụng **EJS** (Embedded JavaScript) làm template engine để render HTML động
- Tích hợp EJS vào kiến trúc MVC
- Sử dụng **partials** (header, footer) để tái sử dụng giao diện
- Phục vụ file tĩnh (CSS, JS, hình ảnh) với `express.static`

Bước 1: Tạo project và cài đặt

```
mkdir 5.2.ejs
cd 5.2.ejs
npm init -y
npm install express ejs
npm install nodemon --save-dev
```

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` vào `package.json`.

Bước 2: Tạo cấu trúc thư mục

```
mkdir -p models controllers routes views/partials views/users public/css public/js
public/images
```

Cấu trúc project:

```
5.2.ejs/
├─ index.js
├─ models/
│   └─ user.model.js
├─ controllers/
│   └─ user.controller.js
├─ routes/
│   └─ user.route.js
├─ views/
│   └─ partials/
│       ├── header.ejs
│       └─ footer.ejs
│   └─ users/
│       ├── index.ejs
│       └─ detail.ejs
└─ public/
    ├── css/
    │   └─ style.css
    ├── js/
    │   └─ main.js
```

```
└─ images/
   └─ logo.png
```

Bước 3: Tạo Model

Tạo file `models/user.model.js` :

```
let users = [
  {
    id: 1,
    name: "tuan",
    age: 20
  },
  {
    id: 2,
    name: "vu",
    age: 21
  }
]

export default users
```

Bước 4: Tạo Controller

Tạo file `controllers/user.controller.js` :

```
import users from "../models/user.model.js";

export const getAll = (req, res) => {
  res.render('users/index', { users })
}

export const getOne = (req, res) => {
  const user = users.find(u => u.id === Number(req.params.id))
  res.render('users/detail', { user })
}
```

Giải thích:

- `res.render('users/index', { users })` - Thay vì trả JSON (`res.json`), bây giờ render template EJS. Tham số thứ 2 là object dữ liệu truyền vào template.
- `{ users }` là viết tắt ES6 của `{ users: users }` .

Bước 5: Tạo Route

Tạo file `routes/user.route.js` :

```
import { Router } from 'express'
import { getAll, getOne } from '../controllers/user.controller.js'

const router = Router();
```

```
router.get('/', getAll)
router.get('/:id', getOne)

export default router
```

Bước 6: Tạo Views (Template EJS)

Tạo file `views/partials/header.ejs` :

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="/css/style.css">
</head>

<body>
  <nav>
    <a href="/users">Trở về danh sách users</a>
  </nav>
  <hr>
```

Tạo file `views/partials/footer.ejs` :

```
<hr>
<footer>Nodejs + EJS</footer>
<script src="/js/main.js"></script>
</body>

</html>
```

Tạo file `views/users/index.ejs` (trang danh sách):

```
<%- include('../partials/header.ejs')%>

<div id="test">DANH SÁCH USERS:</div>
<ul>
  <% users.forEach(user=> { %>
    <li>
      <a href="/users/<%= user.id%>">
        <%= user.name%>
      </a>
    </li>
  <%}) %>
</ul>
```

```
<%- include('../partials/footer.ejs')%>
```

Tạo file `views/users/detail.ejs` (trang chi tiết):

```
<%- include('../partials/header.ejs')%>

THÔNG TIN USER:
<br>
Name: <%= user.name%><br>
Age: <%= user.age%><br>

<%- include('../partials/footer.ejs')%>
```

Cú pháp EJS:

Cú pháp	Ý nghĩa	Ví dụ
<%= ... %>	In giá trị ra HTML (có escape HTML)	<%= user.name %>
<%- ... %>	In giá trị KHÔNG escape (dùng cho include)	<%- include('header.ejs') %>
<% ... %>	Chạy JavaScript (không in ra)	<% users.forEach(...) %>

Bước 7: Tạo file tĩnh

Tạo file `public/css/style.css` :

```
#test {
  color: red;
}
```

Tạo file `public/js/main.js` :

```
alert('from main.js')
```

Thêm 1 file hình bất kỳ vào `public/images/logo.png` (có thể tải logo nhỏ nào đó).

Bước 8: Tạo file chính index.js

Tạo file `index.js` :

```
import express from 'express'
import { fileURLToPath } from 'url'
import path from 'path'
import userRouter from './routes/user.route.js'

const __filename = fileURLToPath(import.meta.url)
const __dirname = path.dirname(__filename)

const app = express()
```

```

app.use(express.json())
app.use(express.static(path.join(__dirname, 'public')))
app.use('/users', userRouter)

app.set('view engine', 'ejs')
app.set('views', path.join(__dirname, 'views'))

app.listen(3000, () => {
  console.log('Server is running ...');
})

```

Giải thích:

- `app.set('view engine', 'ejs')` - Đăng ký EJS làm template engine. Khi gọi `res.render('users/index')`, Express sẽ tự tìm file `views/users/index.ejs`.
- `app.set('views', path.join(__dirname, 'views'))` - Chỉ định thư mục chứa template.
- `app.use(express.static(path.join(__dirname, 'public')))` - Phục vụ file tĩnh. File `public/css/style.css` sẽ truy cập được qua URL `/css/style.css`.
- Cần tạo lại `__dirname` bằng `fileURLToPath + path.dirname` vì ES Module không có sẵn biến này (đã học ở Chương 2).

Bước 9: Chạy và test

```
npm run dev
```

Mở trình duyệt, truy cập: `http://localhost:3000/users`

Kết quả:

- Hiển thị trang HTML với tiêu đề "DANH SÁCH USERS:" (màu đỏ do CSS)
- Danh sách 2 user dạng link: tuan, vu
- Navbar với link "Trở về danh sách users"
- Alert "from main.js" hiện lên (do file JS client)

Click vào tên user (vd: "tuan"), trình duyệt chuyển đến: `http://localhost:3000/users/1`

Kết quả:

- Hiển thị: "THÔNG TIN USER:" - Name: tuan - Age: 20

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Error: Cannot find module 'ejs'	Chưa cài EJS	Chạy <code>npm install ejs</code>
Error: Failed to lookup view "users/index"	Sai đường dẫn views hoặc thiếu <code>app.set('views', ...)</code>	Kiểm tra thư mục <code>views/users/index.ejs</code> tồn tại. Kiểm tra <code>app.set('views', ...)</code> trong <code>index.js</code>
CSS/JS/hình không load được	Thiếu <code>express.static</code> hoặc sai đường dẫn	Đảm bảo có <code>app.use(express.static(...))</code> và file nằm trong thư mục <code>public/</code>

ReferenceError: users is not defined trong EJS	Không truyền dữ liệu vào res.render()	Đảm bảo truyền data: res.render('users/index', { users })
Partial không hiển thị	Sai đường dẫn include	Kiểm tra đường dẫn tương đối: <%- include('../partials/header.ejs') %>

Chương 6: Middleware

Bài 6.2-code-log: Middleware Ghi Log Request

Mục tiêu

- Hiểu middleware là gì và cách hoạt động
- Tạo middleware toàn cục (global middleware)
- Hiểu vai trò của hàm `next()`

Bước 1: Tạo project

```
mkdir 6.2.code-log
cd 6.2.code-log
npm init -y
npm install express
npm install nodemon --save-dev
```

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` vào `package.json`.

Bước 2: Viết code

Tạo file `index.js`:

```
import express from 'express'

const app = express()

app.use((req, res, next) => {
  console.log(`Method: ${req.method} - URL: ${req.url}`);

  next();
})

app.get('/', (req, res) => {
  res.send('Hello world!');
})

app.get('/users', (req, res) => {
  res.send('Lấy thông tin khách hàng.')
})

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Giải thích:

- **Middleware** là hàm chạy TRƯỚC khi request đến route handler. Nó nhận 3 tham số: `req`, `res`, `next`.

- `app.use((req, res, next) => {...})` - Đăng ký middleware toàn cục, chạy cho MỌI request đến server.
- `next()` - Gọi hàm này để chuyển quyền xử lý sang middleware/route tiếp theo. Nếu KHÔNG gọi `next()`, request sẽ bị "treo" - client chờ mãi không nhận được response.

Request → Middleware (log) → next() → Route Handler → Response

Bước 3: Chạy và test

```
npm run dev
```

Truy cập `http://localhost:3000` rồi `http://localhost:3000/users` trên trình duyệt.

Terminal hiển thị:

```
Server is running ...
Method: GET - URL: /
Method: GET - URL: /users
```

Mỗi request đều bị middleware "chặn" và ghi log trước khi đến route handler.

Bài 6.2-code-role-admin: Middleware Kiểm Tra Quyền (Authorization)

Mục tiêu

- Tạo middleware kiểm tra quyền truy cập (authorization)
- Áp dụng middleware cho từng route riêng lẻ (route-specific middleware)
- Trả về HTTP status code phù hợp (401 Unauthorized)

Bước 1: Tạo project

```
mkdir 6.2.code-role-admin
cd 6.2.code-role-admin
npm init -y
npm install express
npm install nodemon --save-dev
```

Thêm `"type": "module"` và script `"dev": "nodemon index.js"`.

Bước 2: Viết code

Tạo file `index.js`:

```
import express from 'express'

const app = express()

const checkAdmin = (req, res, next) => {
  const role = req.query.role;
  if (role === 'admin') {
```

```

    next()
  }
  res.status(401).json({ error: 'Unauthorized' })
}

app.get('/', (req, res) => {
  res.send('Hello world!')
})

app.get('/dashboard', checkAdmin, (req, res) => {
  res.send('Welcome!')
})

app.listen(3000, () => {
  console.log('Server is running ...');
})

```

Giải thích:

- **checkAdmin** - Middleware tự viết, kiểm tra query param `role` có phải `admin` không.
- **Route-specific middleware** - Thay vì dùng `app.use()` (áp dụng cho tất cả), middleware `checkAdmin` chỉ được gắn vào route `/dashboard` bằng cách truyền làm tham số thứ 2 của `app.get()`.
- **res.status(401)** - Trả về mã 401 (Unauthorized) nếu không phải admin.

```

GET /           → Không qua middleware → Response trực tiếp
GET /dashboard → checkAdmin → (nếu admin) → next() → Response
                  → (nếu không) → 401 Unauthorized

```

Bước 3: Chạy và test

```
npm run dev
```

Test 1 - Trang public (không cần quyền):

```
http://localhost:3000
```

Kết quả: `Hello world!`

Test 2 - Truy cập dashboard KHÔNG có quyền:

```
http://localhost:3000/dashboard
```

Kết quả: `{"error":"Unauthorized"}` (status 401)

Test 3 - Truy cập dashboard VỚI quyền admin:

```
http://localhost:3000/dashboard?role=admin
```

Kết quả: `Welcome!`

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Request bị "treo", không có response	Quên gọi <code>next()</code> trong middleware	Luôn gọi <code>next()</code> hoặc gửi response (<code>res.send()</code> , <code>res.json()</code>)
Middleware chạy cho tất cả route	Dùng <code>app.use(middleware)</code> thay vì gắn vào route cụ thể	Dùng <code>app.get('/path', middleware, handler)</code> nếu chỉ muốn áp dụng cho 1 route
Cannot set headers after they are sent	Vừa gọi <code>next()</code> vừa gửi response	Dùng <code>return next()</code> hoặc <code>return res.status(401)...</code> để dừng hàm ngay

Chương 7: MongoDB & Mongoose

Yêu cầu: Cần có tài khoản MongoDB Atlas (miễn phí) tại <https://cloud.mongodb.com>. Tạo cluster và lấy connection string.

Bài 7.3: Giới Thiệu Mongoose - Kết Nối MongoDB

Mục tiêu

- Kết nối Node.js với MongoDB Atlas qua Mongoose
- Định nghĩa Schema và Model
- Thực hiện thao tác Create & Read cơ bản
- Sử dụng biến môi trường với `dotenv`

Bước 1: Tạo project và cài đặt

```
mkdir 7.3.mongoose
cd 7.3.mongoose
npm init -y
npm install express mongoose dotenv
npm install nodemon --save-dev
```

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` vào `package.json`.

Bước 2: Cấu hình biến môi trường

Tạo file `.env`:

```
MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/?
appName=Cluster0
```

Thay `username`, `password` và `cluster0.xxxxx` bằng thông tin MongoDB Atlas của bạn.

Tạo file `.env.example` (mẫu cho người khác):

```
MONGODB_URI=mongodb+srv://username:password@cluster0.takqys0.mongodb.net/?
appName=Cluster0
```

Tạo file `.gitignore`:

```
node_modules
.env
```

Quan trọng: File `.env` chứa mật khẩu database, KHÔNG ĐƯỢC đưa lên Git.

Bước 3: Tạo kết nối database

Tạo thư mục `config` và file `config/db.js`:

```
mkdir config
```

```
import mongoose from 'mongoose'

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI)
    console.log('Kết nối MongoDB thành công.');
```

- **Model** là class để tương tác với collection trong MongoDB.
- `mongoose.model('Student', studentSchema)` tạo collection tên `students` (tự động chuyển sang lowercase + thêm "s").

Bước 5: Tạo file chính

Tạo file `index.js` :

```
import express from 'express'
import dotenv from 'dotenv'
import connectDB from './config/db.js'
import mongoose from 'mongoose'
import Student from './models/student.model.js'

dotenv.config()

connectDB()

const app = express()
app.use(express.json())

app.get('/status', (req, res) => {
  const state = mongoose.connection.readyState
  const map = {
    0: 'Disconnected',
    1: 'Connected',
    2: 'Connecting',
    3: 'Disconnecting',
    99: 'Uninitialized'
  }
  res.json({ db: map[state] })
})

app.get('/api/students', async (req, res) => {
  try {
    const students = await Student.find()
    res.json(students)
  } catch (error) {
    console.log('Lỗi lấy danh sách sinh viên: ', error.message);
    res.json({ error: error.message })
  }
})

app.post('/api/students', async (req, res) => {
  try {
    const student = new Student(req.body)
    await student.save()
    res.status(201).json(student)
  } catch (error) {
    console.log('Lỗi thêm sinh viên: ', error.message);
    res.json({ error: error.message })
  }
})
```

```
})

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Giải thích:

- **dotenv.config()** - Đọc file `.env` và nạp các biến vào `process.env`.
- **connectDB()** - Gọi hàm kết nối database đã tạo ở bước 3.
- **Student.find()** - Lấy tất cả documents từ collection `students`.
- **new Student(req.body) + student.save()** - Tạo document mới từ dữ liệu request body và lưu vào database.
- **res.status(201)** - HTTP 201 (Created) cho biết tạo mới thành công.
- **/status** - Endpoint kiểm tra trạng thái kết nối database.

Bước 6: Chạy và test

```
npm run dev
```

Test kiểm tra kết nối:

```
curl http://localhost:3000/status
```

Kết quả: `{"db": "Connected"}`

Test tạo sinh viên:

```
curl -X POST http://localhost:3000/api/students \
-H "Content-Type: application/json" \
-d '{"name": "Nguyen Van A", "studentId": "SV001", "gpa": 8.5}'
```

Kết quả:

```
{
  "_id": "664...",
  "name": "Nguyen Van A",
  "studentId": "SV001",
  "major": "Công nghệ thông tin",
  "gpa": 8.5,
  "__v": 0
}
```

Lưu ý: `major` tự động được gán giá trị mặc định, `_id` do MongoDB tự tạo.

Test lấy danh sách:

```
curl http://localhost:3000/api/students
```


Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
MongoServerError: bad auth	Sai username/password MongoDB	Kiểm tra lại thông tin trong .env
MongoNetworkError: connect ECONNREFUSED	Không kết nối được MongoDB Atlas	Kiểm tra kết nối internet. Vào MongoDB Atlas -> Network Access -> thêm IP 0.0.0.0/0 (cho phép tất cả)
MongooseError: Operation ... timed out	Kết nối quá chậm/timeout	Kiểm tra internet, hoặc whitelist IP trong MongoDB Atlas
E11000 duplicate key error	studentId bị trùng (đã tồn tại)	Dùng studentId khác, hoặc xóa document cũ
ValidationError: name: Path 'name' is required	Thiếu field bắt buộc	Đảm bảo gửi đủ các field có required: true
process.env.MONGODB_URI là undefined	Thiếu dotenv.config() hoặc file .env	Kiểm tra đã gọi dotenv.config() và file .env tồn tại

Bài 7.4: Thực Hành - CRUD Product Với Mongoose (MVC)

Mục tiêu

- Xây dựng REST API CRUD hoàn chỉnh kết nối MongoDB thật
- Áp dụng kiến trúc MVC với Mongoose
- Sử dụng các Mongoose methods: `find`, `findById`, `findByIdAndUpdate`, `findByIdAndDelete`
- Option `timestamps` tự động theo dõi thời gian tạo/cập nhật

Bước 1: Tạo project

```
mkdir 7.4.thuchanh-product
cd 7.4.thuchanh-product
npm init -y
npm install express mongoose dotenv
npm install nodemon --save-dev
```

Thêm `"type": "module"` và script `"dev": "nodemon index.js"`.

Cấu hình `.env`, `.env.example`, `.gitignore` giống bài 7.3.

```
mkdir config models controllers routes
```

Cấu trúc project:

```
7.4.thuchanh-product/
├─ index.js
├─ .env
```

```
|— config/
|   └─ db.js
|— models/
|   └─ product.model.js
|— controllers/
|   └─ product.controller.js
└─ routes/
    └─ product.route.js
```

Bước 2: Tạo kết nối database

Tạo file `config/db.js` :

```
import mongoose from "mongoose";

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI)
    console.log('Kết nối MongoDB thành công');
  } catch (error) {
    console.error('Lỗi kết nối MongoDB: ', error.message)
    process.exit()
  }
}

export default connectDB
```

`process.exit()` - Dừng ứng dụng nếu không kết nối được database (vì không thể hoạt động nếu không có DB).

Bước 3: Tạo Model

Tạo file `models/product.model.js` :

```
import mongoose from 'mongoose'

const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: { type: Number, min: 0 },
  category: { type: String },
  inStock: { type: Boolean, default: true }
}, { timestamps: true })

const Product = mongoose.model('Product', productSchema)

export default Product
```

`{ timestamps: true }` - Mongoose tự động thêm 2 field:

- `createdAt` - Thời điểm tạo document
- `updatedAt` - Thời điểm cập nhật gần nhất

Bước 4: Tạo Controller

Tạo file `controllers/product.controller.js` :

```
import Product from "../models/product.model.js";

export const getAll = async (req, res) => {
  try {
    const products = await Product.find()
    res.json(products)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

export const getOne = async (req, res) => {
  try {
    const product = await Product.findById(req.params.id)
    if (!product) {
      return res.status(404).json({ error: 'Not found' })
    }
    res.json(product)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

export const create = async (req, res) => {
  try {
    const product = new Product(req.body)
    await product.save()
    res.status(201).json(product)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

export const update = async (req, res) => {
  try {
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    )
    if (!product) {
      return res.status(404).json({ error: 'Not found' })
    }
    res.json(product)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}
```

```
export const remove = async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id)
    if (!product) {
      return res.status(404).json({ error: 'Not found' })
    }
    res.json(product)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}
```

Giải thích các Mongoose methods:

Method	Ý nghĩa	Ghi chú
Product.find()	Lấy tất cả documents	Trả về mảng
Product.findById(id)	Tìm 1 document theo _id	Trả về object hoặc null
new Product(data) + .save()	Tạo và lưu document mới	Chạy validation
Product.findByIdAndUpdate(id, data, options)	Tìm và cập nhật	new: true trả về document sau khi cập nhật
Product.findByIdAndDelete(id)	Tìm và xóa	Trả về document đã xóa hoặc null

- **{ new: true }** - Mặc định `findByIdAndUpdate` trả về document CŨ (trước khi update). Thêm option này để trả về document MỚI.
- **{ runValidators: true }** - Chạy validation schema khi update (mặc định không chạy).

Bước 5: Tạo Route

Tạo file `routes/product.route.js` :

```
import { Router } from "express";
import { getAll, getOne, create, update, remove } from
"../controllers/product.controller.js";

const router = Router()

router.get('/', getAll)
router.get('/:id', getOne)
router.post('/', create)
router.put('/:id', update)
router.delete('/:id', remove)
```

```
export default router
```

Bước 6: Tạo file chính

Tạo file `index.js` :

```
import express from 'express'
import dotenv from 'dotenv'
import mongoose from 'mongoose'
import connectDB from './config/db.js'
import productRouter from './routes/product.route.js'

dotenv.config()

const app = express()
app.use(express.json())

await connectDB()

app.use('/api/products', productRouter)

app.get('/status', (req, res) => {
  const state = mongoose.connection.readyState
  const map = {
    0: 'Disconnected',
    1: 'Connected',
    2: 'Connecting',
    3: 'Disconnecting',
    99: 'uninitialized'
  }
  res.json({ db: map[state] })
})

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

`await connectDB()` - Chờ kết nối database thành công TRƯỚC khi server bắt đầu nhận request.

Bước 7: Chạy và test

```
npm run dev
```

```
# Tạo product
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Laptop", "price": 1500, "category": "Electronics"}'

# Lấy tất cả
```

```
curl http://localhost:3000/api/products

# Lấy theo id (thay <id> bằng _id thật)
curl http://localhost:3000/api/products/<id>

# Cập nhật
curl -X PUT http://localhost:3000/api/products/<id> \
  -H "Content-Type: application/json" \
  -d '{"price": 1200}'

# Xóa
curl -X DELETE http://localhost:3000/api/products/<id>
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
CastError: Cast to ObjectId failed	ID không đúng format MongoDB ObjectId (24 ký tự hex)	Dùng đúng _id từ response khi tạo product
Dữ liệu vẫn còn sau restart	Khác với các bài trước, giờ dữ liệu lưu trong MongoDB thật	Đây là hành vi đúng - dữ liệu persistent
timestamps không xuất hiện	Thiếu { timestamps: true } trong schema	Thêm option { timestamps: true } làm tham số thứ 2 của new mongoose.Schema()

Chương 9: MongoDB Nâng Cao

Bài 9.4: Query Operators - Toán Tử Truy Vấn MongoDB

Mục tiêu

- Sử dụng các toán tử truy vấn MongoDB: `$gt` , `$in` , `$and` , `$regex`
- Schema với nested object và mảng
- Seed dữ liệu mẫu vào database

Bước 1: Tạo project

```
mkdir 9.4.mongo-operators
cd 9.4.mongo-operators
npm init -y
npm install express mongoose dotenv
npm install nodemon --save-dev
```

Cấu hình "type": "module" , script "dev" , `.env` , `.gitignore` như các bài trước.

```
mkdir config models controllers routes
```

Bước 2: Tạo kết nối database

File `config/db.js` giống bài 7.3.

Bước 3: Tạo Model với nested object & mảng

Tạo file `models/product.model.js` :

```
import mongoose from "mongoose";

const productSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, index: true },
    price: { type: Number, min: 0 },
    category: { type: String },
    inStock: { type: Boolean, default: true },
    address: {
      city: { type: String },
      country: { type: String }
    },
    tags: [{ type: String }]
  }
)

productSchema.index({ inStock: 1, name: -1 })

const Product = mongoose.model('Product', productSchema)
```

```
export default Product
```

Giải thích:

- **address** - Nested object (object lồng nhau) chứa **city** và **country** .
- **tags: [{ type: String }]** - Mảng các string.
- **index: true** - Tạo index trên field **name** để tìm kiếm nhanh hơn.
- **productSchema.index({ inStock: 1, name: -1 })** - Tạo compound index (index kết hợp): **inStock** tăng dần, **name** giảm dần.

Bước 4: Tạo Controller với Seed & Query

Tạo file `controllers/product.controllers.js` :

```
import Product from "../models/product.model.js";

export const seed = async (req, res) => {
  try {
    await Product.deleteMany()
    await Product.insertMany([
      { name: "iPhone 15", price: 999, category: "Electronics", inStock: true,
address: { city: "hanoi", country: "vietnam" }, tags: ["sale", "new"] },
      { name: "iPhone 14", price: 799, category: "Electronics", inStock:
false, address: { city: "hanoi", country: "vietnam" }, tags: ["sale"] },
      { name: "Laptop pro", price: 1500, category: "Electronics", inStock:
true, address: { city: "hcm", country: "vietnam" }, tags: ["sale", "new"] },
      { name: "Keyboard", price: 100, category: "Accessories", inStock: true,
tags: ["sale"] },
      { name: "Mouse", price: 50, category: "Accessories", inStock: false },
    ])
    res.json({ message: 'Seed data xong' })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

export const test = async (req, res) => {
  try {
    const products = await Product.find(
      // { price: { $gt: 799 } }
      // { category: { $in: ['Electronics', 'Accessories'] } }
      // {
      //   $and: [
      //     { price: { $gt: 100 } },
      //     { inStock: true }
      //   ]
      // }
      { name: { $regex: "iPhone", $options: 'i' } }
    )
    res.json(products)
  } catch (error) {
```



```

    res.status(500).json({ message: error.message })
  }
}

```

Giải thích các Query Operators:

Operator	Ý nghĩa	Ví dụ	Kết quả
\$gt	Greater than (lớn hơn)	{ price: { \$gt: 799 } }	Sản phẩm giá > 799
\$in	Nằm trong danh sách	{ category: { \$in: ['Electronics'] } }	Sản phẩm thuộc Electronics
\$and	Và (tất cả điều kiện đúng)	{ \$and: [{price: {\$gt:100}}, {inStock: true}] }	Giá > 100 VÀ còn hàng
\$regex	Tìm theo pattern	{ name: { \$regex: "iPhone", \$options: 'i' } }	Tên chứa "iphone" (không phân biệt hoa/thường)

- **\$options: 'i'** - Case-insensitive (không phân biệt chữ hoa/thường).
- **Product.deleteMany()** - Xóa tất cả documents trong collection.
- **Product.insertMany([...])** - Chèn nhiều documents cùng lúc.

Bước 5: Tạo Route và file chính

Tạo file `routes/product.route.js` :

```

import { Router } from 'express';
import { seed, test } from '../controllers/product.controllers.js';

const router = Router()

router.post('/seed', seed)
router.get('/test', test)

export default router

```

Tạo file `index.js` :

```

import express from 'express'
import dotenv from 'dotenv'
import productRouter from './routes/product.route.js'
import connectDB from './config/db.js'

dotenv.config()
await connectDB()

const app = express()
app.use(express.json())

app.use('/api/products', productRouter)

```

```
app.listen(3000, () => {  
  console.log('Server is running ...');  
})
```

Bước 6: Chạy và test

```
npm run dev
```

```
# Seed dữ liệu mẫu (chạy 1 lần)  
curl -X POST http://localhost:3000/api/products/seed  
  
# Test query operators  
curl http://localhost:3000/api/products/test
```

Thử thay đổi query trong hàm `test()` bằng cách bỏ comment từng operator để xem kết quả khác nhau.

Bài 9.th: Thực Hành - Explain Query, Index & Phân Trang

Mục tiêu

- Seed 100,000 documents để test hiệu năng
- So sánh tốc độ query CÓ index vs KHÔNG có index bằng `explain()`
- Thực hiện phân trang (pagination) với `skip`, `limit`, `sort`

Bước 1: Tạo project

```
mkdir 9.thuc-hanh  
cd 9.thuc-hanh  
npm init -y  
npm install express mongoose dotenv  
npm install nodemon --save-dev
```

Cấu hình tương tự các bài trước. Tạo `config/db.js` giống bài 7.3.

```
mkdir config models controllers routes
```

Bước 2: Tạo Model với duplicate fields (cho so sánh)

Tạo file `models/product.model.js` :

```
import mongoose from 'mongoose';  
  
const productSchema = new mongoose.Schema({  
  name: { type: String, required: true },  
  category: { type: String },  
  price: { type: String, min: 0 },  
  inStock: { type: Boolean },
```

```

    name2: { type: String, required: true },
    category2: { type: String },
  }, { timestamps: true })

productSchema.index({ name: 1, category: -1 })

const Product = mongoose.model('Product', productSchema)

export default Product

```

`name2` , `category2` là bản sao KHÔNG có index - dùng để so sánh tốc độ với `name` , `category` CÓ index.

Bước 3: Tạo Controller

Tạo file `controllers/product.controller.js` :

```

import Product from '../models/product.model.js';

export const seed = async (req, res) => {
  try {
    await Product.deleteMany()
    await Product.collection.dropIndexes()

    const bulk = []
    for (let i = 0; i < 100000; i++) {
      const category = i % 2 === 0 ? 'Electronics' : 'Accessories';
      const name = `Product ${i}`
      bulk.push({
        name,
        category,
        name2: name,
        category2: category,
        price: Math.floor(Math.random() * 100000),
        inStock: i % 3 === 0
      })
    }
    await Product.insertMany(bulk)

    res.json({ message: 'Đã thực hiện seed 100000 documents xong' })
  } catch (error) {
  }
}

export const explainQuery = async (req, res) => {
  try {
    await Product.collection.dropIndexes()
    await Product.collection.createIndex({ name: 1, category: -1 })

    const before = await Product

```

```

        .find({ category2: 'Electronics', name2: /^Product/ })
        .explain('executionStats')

const after = await Product
    .find({ category: 'Electronics', name: /^Product/ })
    .explain('executionStats')

res.json({
  without_index: {
    stage: before.executionStats.executionStages.stage,
    docsExamined: before.executionStats.totalDocsExamined,
    docsReturned: before.executionStats.totalDocsReturned,
    executionTimeMillis: before.executionStats.executionTimeMillis,
  },
  with_index: {
    stage: after.executionStats.executionStages.stage,
    docsExamined: after.executionStats.totalDocsExamined,
    docsReturned: after.executionStats.totalDocsReturned,
    executionTimeMillis: after.executionStats.executionTimeMillis,
  }
})
} catch (error) {
  res.status(500).json({ error: error.message })
}
}

export const paginate = async (req, res) => {
  try {
    const page = Math.max(1, parseInt(req.query.page) || 1)
    const limit = Math.max(1, parseInt(req.query.limit) || 10)
    const skip = (page - 1) * limit

    const sortField = req.query.sort || 'createdAt'
    const sortOrder = req.query.order === 'asc' ? 1 : -1
    const sort = { [sortField]: sortOrder }

    const filter = req.query.name ? { name: { $regex: req.query.name, $options:
'i' } } : {}

    const [products, total] = await Promise.all([
      Product.find(filter).sort(sort).skip(skip).limit(limit),
      Product.countDocuments(filter)
    ])

    res.json({
      data: products,
      pagination: {
        total,
        page,
        limit,
        totalPages: Math.ceil(total / limit),
        hasNext: page < Math.ceil(total / limit),

```

```

        hasPrev: page > 1,
      }
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

```

Giải thích hàm `explainQuery` :

- `.explain('executionStats')` - Yêu cầu MongoDB trả về thống kê thực thi thay vì dữ liệu.
- **COLLSCAN** - Collection Scan, quét TOÀN BỘ documents (chậm).
- **IXSCAN** - Index Scan, quét qua index (nhận).
- So sánh `docsExamined` giữa 2 cách: không index phải quét ~100,000 docs, có index chỉ quét vài nghìn.

Giải thích hàm `paginate` :

Tham số	Ý nghĩa	Mặc định
page	Trang hiện tại	1
limit	Số items mỗi trang	10
sort	Field sắp xếp	createdAt
order	Thứ tự (asc/desc)	desc
name	Lọc theo tên (regex)	Không lọc

- `.skip(skip)` - Bỏ qua `skip` documents đầu tiên.
- `.limit(limit)` - Lấy tối đa `limit` documents.
- `.sort(sort)` - Sắp xếp kết quả.
- `Promise.all([...])` - Chạy song song query lấy data và đếm tổng số documents.

Bước 4: Tạo Route và file chính

Tạo file `routes/product.route.js` :

```

import { Router } from "express";
import { explainQuery, paginate, seed } from "../controllers/product.controller.js";

const router = new Router();

router.post('/seed', seed)
router.get('/explain', explainQuery)
router.get('/paginate', paginate)

export default router

```

Tạo file `index.js` :

```
import express from 'express';
import connectDB from './config/db.js';
import dotenv from 'dotenv'
import productRouter from './routes/product.route.js';

dotenv.config()
await connectDB()
const app = express()
app.use(express.json())

app.use('/api/products', productRouter)

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Bước 5: Chạy và test

```
npm run dev
```

```
# Seed 100,000 documents (có thể mất vài giây)
curl -X POST http://localhost:3000/api/products/seed

# So sánh hiệu năng index
curl http://localhost:3000/api/products/explain

# Phân trang: trang 1, mỗi trang 5
curl "http://localhost:3000/api/products/paginate?page=1&limit=5"

# Phân trang + lọc theo tên + sắp xếp theo giá tăng dần
curl "http://localhost:3000/api/products/paginate?page=1&limit=5&name=Product%201&sort=price&order=asc"
```

Kết quả explain (mẫu):

```
{
  "without_index": {
    "stage": "COLLSCAN",
    "docsExamined": 100000,
    "docsReturned": 50000
  },
  "with_index": {
    "stage": "IXSCAN",
    "docsExamined": 50000,
    "docsReturned": 50000
  }
}
```

Không có index: quét 100,000 docs. Có index: quét ít hơn đáng kể.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Seed quá lâu hoặc timeout	100,000 documents cần thời gian	Chờ đợi, hoặc giảm số lượng xuống 10,000 để test
explain trả về lỗi	Structure của explain result thay đổi theo phiên bản MongoDB	Kiểm tra phiên bản MongoDB Atlas đang dùng

Chương 10: Xác Thực & Phiên Làm Việc

Bài 10.2: Mã Hóa Mật Khẩu Với Bcrypt

Mục tiêu

- Hiểu tại sao KHÔNG ĐƯỢC lưu mật khẩu dạng plain text
- Sử dụng `bcrypt` để hash mật khẩu khi đăng ký
- Sử dụng `bcrypt.compare()` để xác thực khi đăng nhập

Bước 1: Tạo project

```
mkdir 10.2-bcrypt
cd 10.2-bcrypt
npm init -y
npm install express mongoose bcrypt dotenv
npm install nodemon --save-dev
```

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` .

Cấu hình `.env` :

```
PORT=3000
MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/?
appName=Cluster0
```

```
mkdir config models controllers routes
```

Bước 2: Tạo kết nối database

File `config/db.js` :

```
import mongoose from "mongoose";

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log("MongoDB connected");
  } catch (error) {
    console.error("Error connecting to MongoDB:", error);
    process.exit(1);
  }
};

export default connectDB;
```

Bước 3: Tạo User Model

Tạo file `models/user.model.js` :


```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true
  },
  hashedPassword: {
    type: String,
    required: true
  }
}, { timestamps: true });

const User = mongoose.model("User", userSchema);

export default User;
```

Lưu ý: Field tên là `hashedPassword` - nhắc nhở rằng KHÔNG BAO GIỜ lưu mật khẩu gốc.

Bước 4: Tạo Auth Controller

Tạo file `controllers/auth.controller.js` :

```
import bcrypt from "bcrypt";
import User from "../models/user.model.js";

export const registerUser = async (req, res) => {
  try {
    const { username, password } = req.body;
    const hashedPassword = await bcrypt.hash(password, 10);
    const user = new User({ username, hashedPassword });
    await user.save();
    res.status(201).json({ message: "User registered successfully" });
  } catch (error) {
    res.status(500).json({ message: "Error registering user" });
  }
};

export const loginUser = async (req, res) => {
  try {
    const { username, password } = req.body;
    const user = await User.findOne({ username });
    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }
    const isPasswordValid = await bcrypt.compare(password, user.hashedPassword);
    if (!isPasswordValid) {
      return res.status(401).json({ message: "Invalid password" });
    }
    res.status(200).json({ message: "User logged in successfully" });
  }
};
```

```

    } catch (error) {
      res.status(500).json({ message: "Error logging in user" });
    }
  };
};

```

Giải thích Bcrypt:

- **bcrypt.hash(password, 10)** - Hash mật khẩu với 10 **salt rounds**. Salt là chuỗi ngẫu nhiên thêm vào trước khi hash, số 10 là độ phức tạp (càng cao càng an toàn nhưng càng chậm).
 - Input: "myPassword123" -> Output: "\$2b\$10\$xK8f3g..." (chuỗi hash dài ~60 ký tự)
 - Cùng một mật khẩu, mỗi lần hash cho ra kết quả KHÁC NHAU (nhờ salt ngẫu nhiên).
- **bcrypt.compare(password, hashedPassword)** - So sánh mật khẩu gốc với hash đã lưu. Trả về true nếu khớp, false nếu không. KHÔNG CẦN biết salt vì salt được lưu trong chuỗi hash.

Bước 5: Tạo Route

Tạo file `routes/auth.route.js` :

```

import { Router } from "express";
import { registerUser, loginUser } from "../controllers/auth.controller.js";

const router = Router();

router.post("/register", registerUser);
router.post("/login", loginUser);

export default router;

```

Bước 6: Tạo file chính

Tạo file `index.js` :

```

import express from "express";
import dotenv from "dotenv";
import connectDB from "../config/db.js";
import authRoute from "../routes/auth.route.js";

dotenv.config();
await connectDB();

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use("/api/auth", authRoute);

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});

```

Bước 7: Chạy và test

```
npm run dev
```

Test đăng ký:

```
curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d '{"username": "student01", "password": "123456"}'
```

Kết quả: `{"message": "User registered successfully"}`

Kiểm tra trong MongoDB Atlas: mật khẩu lưu dạng `$2b$10$xK8f3g...` chứ KHÔNG PHẢI `123456`.

Test đăng nhập đúng:

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username": "student01", "password": "123456"}'
```

Kết quả: `{"message": "User logged in successfully"}`

Test đăng nhập sai mật khẩu:

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username": "student01", "password": "wrong"}'
```

Kết quả: `{"message": "Invalid password"}` (status 401)

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Error: Cannot find module 'bcrypt'	Chưa cài bcrypt	npm install bcrypt
Lỗi cài bcrypt trên Windows	bcrypt cần build native module	Thử npm install bcryptjs (pure JS, không cần build) rồi đổi import bcrypt from "bcryptjs"
E11000 duplicate key error	Username đã tồn tại	Dùng username khác

Bài 10.3: Session & Middleware Xác Thực

Mục tiêu

- Quản lý phiên đăng nhập (session) với `express-session`
- Lưu session vào MongoDB với `connect-mongo`
- Tạo middleware `requireLogin` bảo vệ route

- Luồng hoàn chỉnh: Đăng ký -> Đăng nhập -> Truy cập trang bảo vệ -> Đăng xuất

Bước 1: Tạo project

```
mkdir 10.3.session-middleware
cd 10.3.session-middleware
npm init -y
npm install express mongoose bcrypt dotenv express-session connect-mongo
npm install nodemon --save-dev
```

Thêm vào `package.json` :

```
{
  "type": "module",
  "scripts": {
    "dev": "nodemon src/index.js"
  }
}
```

Lưu ý: Script chạy `src/index.js` vì bài này tổ chức code trong thư mục `src/`.

Cấu hình `.env` :

```
MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/?
appName=Cluster0
SESSION_SECRET=my_session_secret
```

```
mkdir -p src/config src/models src/controllers src/middlewares src/routes
```

Cấu trúc project:

```
10.3.session-middleware/
├─ package.json
├─ .env
└─ src/
    ├─ index.js
    ├─ config/
    │   └─ db.js
    │   └─ session.js
    ├─ models/
    │   └─ user.model.js
    ├─ controllers/
    │   └─ user.controller.js
    ├─ middlewares/
    │   └─ requireLogin.js
    └─ routes/
        └─ user.route.js
```

Bước 2: Tạo kết nối database

Tạo file `src/config/db.js` :

```
import mongoose from 'mongoose';

const connectDb = async () => {
  try {
    const conn = mongoose.connect(process.env.MONGODB_URI)
    console.log('Kết nối mongodb thành công.');
```

```
  } catch (error) {
    console.error('Lỗi kết nối mongodb: ', error.message);
  }
}

export default connectDb
```

Bước 3: Cấu hình Session

Tạo file `src/config/session.js` :

```
import session from 'express-session';
import dotenv from 'dotenv';
import MongoStore from 'connect-mongo';
dotenv.config()

const sessionConfig = session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    maxAge: 24 * 60 * 60 * 1000
  },
  store: MongoStore.create({
    mongoUrl: process.env.MONGODB_URI,
    collectionName: 'sessions',
    ttl: 24 * 60 * 60,
    autoRemove: 'native'
  })
})

export default sessionConfig
```

Giải thích cấu hình Session:

Option	Giá trị	Ý nghĩa
secret	Chuỗi bí mật	Dùng để ký (sign) session ID, chống giả mạo
resave	false	Không lưu lại session nếu không thay đổi
saveUninitialized	false	Không tạo session cho request chưa đăng nhập

cookie.httpOnly	true	Cookie chỉ gửi qua HTTP, JavaScript phía client KHÔNG đọc được (chống XSS)
cookie.maxAge	86400000 (24h)	Cookie hết hạn sau 24 giờ
store	MongoStore	Lưu session vào MongoDB thay vì bộ nhớ server
ttl	86400 (24h)	Thời gian sống của session trong MongoDB
autoRemove	'native'	MongoDB tự xóa session hết hạn

Tại sao lưu session vào MongoDB? Mặc định `express-session` lưu trong bộ nhớ RAM - nếu restart server thì mất hết session. Lưu vào MongoDB giúp session tồn tại qua các lần restart.

Bước 4: Tạo User Model

Tạo file `src/models/user.model.js` :

```
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema({
  username: {type: String, require: true, unique: true, trim: true, minlength: [3, "username phải có ít nhất 3 kí tự"]},
  hashedPassword: {type: String, require: true}
}, {timestamps: true})

const User = mongoose.model('User', userSchema)

export default User
```

Bước 5: Tạo Middleware requireLogin

Tạo file `src/middlewares/requireLogin.js` :

```
const requireLogin = (req, res, next) => {
  console.log('==== requireLogin middleware ====');
  console.log('SessionID: ', req.sessionID);
  console.log('User: ', req.session.user ?? 'Chưa đăng nhập');

  if (!req.session.user) {
    return res.status(401).json({error: "Chưa đăng nhập"})
  }
  req.user = req.session.user;
  next();
}

export default requireLogin
```

Giải thích:

- Kiểm tra `req.session.user` - nếu không có nghĩa là chưa đăng nhập.

- `req.user = req.session.user` - Gắn thông tin user vào `req` để các controller sau có thể sử dụng.
- `??` - Nullish coalescing operator, trả về về phải nếu về trái là `null` / `undefined` .

Bước 6: Tạo Controller

Tạo file `src/controllers/user.controller.js` :

```
import bcrypt from 'bcrypt';
import User from '../models/user.model.js';

export const registerUser = async (req, res) => {
  try {
    const {username, password} = req.body

    // kiểm tra username đã tồn tại chưa?
    const existingUser = await User.findOne({username})
    if (existingUser) {
      return res.status(409).json({error: "Username đã tồn tại"})
    }

    // lưu vào mongodb
    const hashedPassword = await bcrypt.hash(password, 10)
    const newUser = await User.create({username, hashedPassword})
    res.status(201).json({
      message: "Đăng kí thành công",
      userId: newUser._id
    })

  } catch (error) {
    console.error('Lỗi đăng ký user: ', error.message);
    res.status(500).json({error: 'Lỗi hệ thống đăng kí user'})
  }
}

export const loginUser = async (req, res) => {
  try {
    const {username, password} = req.body
    const user = await User.findOne({username})
    const storedHash = user.hashedPassword
    if (!storedHash) {
      return res.status(401).json({message: 'Sai thông tin đăng nhập'})
    }

    const isMatch = await bcrypt.compare(password, storedHash)
    if (isMatch) {
      req.session.user = {
        userId: user._id,
        username: user.username
      }
      res.status(200).json({message: 'Đăng nhập thành công', user:
req.session.user, sessionId: req.sessionID})
    }
  }
}
```

```

    }
    else {
      res.status(401).json({message: "Sai thông tin đăng nhập"})
    }
  } catch (error) {
    console.error('Lỗi đăng nhập');
    res.status(500).json({error: 'Lỗi hệ thống đăng nhập'})
  }
}

export const getProfile = async (req, res) => {
  try {
    const user = await User.findById(req.user.userId)

    res.json({
      message: "Lấy profile thành công",
      sessionID: req.sessionID,
      user
    })
  } catch (error) {
    res.status(500).json({error: "Lỗi hệ thống"})
  }
}

export const logout = async (req, res) => {
  const username = req.session.user?.username;
  req.session.destroy((err) => {
    if (err) {
      return res.status(500).json({message: "Lỗi đăng xuất"})
    }
    res.clearCookie('connect.sid')
    res.json({message: `${username} đã đăng xuất!`})
  })
}

```

Giải thích luồng Session:

1. **Đăng ký** - Hash mật khẩu, lưu user vào DB. Kiểm tra trùng username trước (status 409 Conflict).
2. **Đăng nhập** - So sánh mật khẩu bằng `bcrypt.compare()` . Nếu đúng, lưu thông tin user vào `session: req.session.user = { userId, username }` . Session được tự động lưu vào MongoDB qua `connect-mongo` .
3. **Xem profile** - Middleware `requireLogin` kiểm tra session. Nếu đã đăng nhập, lấy thông tin user từ DB.
4. **Đăng xuất** - `req.session.destroy()` xoá session khỏi MongoDB.
`res.clearCookie('connect.sid')` xoá cookie trên trình duyệt.

Bước 7: Tạo Route

Tạo file `src/routes/user.route.js` :

```

import {Router} from 'express';
import { getProfile, loginUser, logout, registerUser } from

```



```

'../controllers/user.controller.js';
import requireLogin from '../middlewares/requireLogin.js';

const router = Router();

router.post('/register', registerUser)
router.post('/login', loginUser)
router.get('/profile', requireLogin, getProfile)
router.post('/logout', requireLogin, logout)

export default router;

```

`/profile` và `/logout` được bảo vệ bởi `requireLogin` - chỉ user đã đăng nhập mới truy cập được.

Bước 8: Tạo file chính

Tạo file `src/index.js` :

```

import express from 'express';
import dotenv from 'dotenv'
import connectDb from './config/db.js';
import userRoute from './routes/user.route.js'
import sessionConfig from './config/session.js';

dotenv.config()
await connectDb()

const app = express()
app.use(express.json())
app.use(sessionConfig)

app.use('/users', userRoute)

app.listen(3000, () => {
  console.log('Server is running ...');
})

```

Bước 9: Chạy và test

```
npm run dev
```

Luồng test hoàn chỉnh:

```

# 1. Đăng ký
curl -X POST http://localhost:3000/users/register \
  -H "Content-Type: application/json" \
  -d '{"username": "student01", "password": "123456"}'

# 2. Đăng nhập (lưu cookie vào file)
curl -X POST http://localhost:3000/users/login \

```

```
-H "Content-Type: application/json" \  
-d '{"username": "student01", "password": "123456"}' \  
-c cookies.txt
```

```
# 3. Xem profile (gửi kèm cookie)  
curl http://localhost:3000/users/profile -b cookies.txt
```

```
# 4. Đăng xuất  
curl -X POST http://localhost:3000/users/logout -b cookies.txt
```

```
# 5. Thử xem profile sau khi đăng xuất (sẽ bị từ chối)  
curl http://localhost:3000/users/profile -b cookies.txt
```

Giải thích flag curl:

- `-c cookies.txt` - Lưu cookie (session ID) vào file.
- `-b cookies.txt` - Gửi cookie từ file đi kèm request.

Kết quả mong đợi:

Bước 3 (profile khi đã đăng nhập):

```
{"message":"Lấy profile thành công","sessionID":"...","user":  
{"_id":"...","username":"student01"}}
```

Bước 5 (profile sau đăng xuất):

```
{"error":"Chưa đăng nhập"}
```

Nếu test bằng Postman: Postman tự quản lý cookie, không cần file `cookies.txt`.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Error: secret option required for sessions	Thiếu SESSION_SECRET trong <code>.env</code>	Thêm SESSION_SECRET=chuoai_bi_mat vào file <code>.env</code>
Cannot find module 'connect-mongo'	Chưa cài	<code>npm install connect-mongo</code>
Profile trả về 401 dù đã login	Không gửi cookie kèm request	Dùng <code>-b cookies.txt</code> với curl hoặc bật "Send cookies" trong Postman
Session mất sau restart	Kiểm tra MongoStore có kết nối đúng	Kiểm tra <code>mongoUrl</code> trong session config trỏ đúng MongoDB Atlas
req.session.user là undefined	Session chưa được tạo (chưa đăng nhập) hoặc cookie hết hạn	Đăng nhập lại